

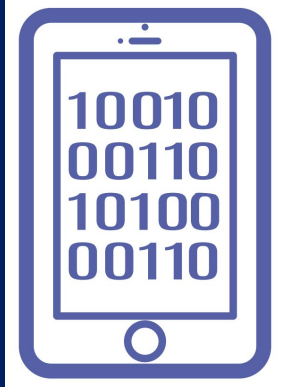
MOBİL UYGULAMA



İSTANBUL DENEYAP TEKNOLOJİ ATÖLYESİ

Ders Saatleri : 09.00 - 13.00 - Lise Sabah Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)

Ders Saatleri : 13.30 - 17.30 - Orta Öğle Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)



MOBİL UYGULAMA DERSİ

3.4.5.Ders: Mobil Uygulamalar-Flutter-Dart



T3 DENEYAP EĞİTMENİ (Hazırlayan ve Sunan): Muhammed Emin Korkunç

(Bilgi Yazar Mühendisi)

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunc-100ba2215 | github.com/muhammedkorkunc

MOBİL UYGULAMA

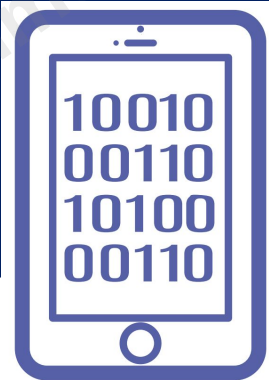


Sabah Grubu

- 09.00 - 09.50: 1. Ders
- 09.50 - 10.00: 1. Mola
- 10.00 - 10.50: 2. Ders
- 10.50 - 11.00: 2. Mola
- 11.00 - 11.50: 3. Ders
- 11.50 - 12.00: 3. Mola
- 12.00 - 13.00: 4. Ders

Öğle Grubu

- 13.30 - 14.20: 1. Ders
- 14.20 - 14.30: 1. Mola
- 14.30 - 15.20: 2. Ders
- 15.20 - 15.30: 2. Mola
- 15.30 - 16.20: 3. Ders
- 16.20 - 16.30: 3. Mola
- 16.30 - 17.30: 4. Ders



1- TERMINAL

KURULUM VİDEOSU

WINDOWS İÇİN=https://www.youtube.com/watch?v=zdj7Np_gzGw

MACBOOK İÇİN=https://youtu.be/VXswG6ZIZsM?si=B7qt3o5Hlcb_T87y

TERMINAL (flutter_console.bat dosyası içinde)

KOD= flutter doctor –android-licenses

KOD= flutter doctor

KOD= flutter create app

KOD= cd app

KOD= flutter run

 Tek yapmanız gereken, terminalde oluşturduğumuz app1 klasörünü tutup VSCode ekranının üzerine sürükleyip bırakmak!

 Sol taraftaki Explorer (Dosya Gezgini) butonuna tıkladığımızda projenin tüm dalları önümüze serilecek. Bizim için en en önemli dosya hangisi biliyor musunuz? Tabii ki lib klasörünün altındaki main.dart dosyası! Tüm iş bu dosyada dönüyor."

APK YAPIMI

kod : flutter build apk –release

C:\Users\DELL\oyun\build\app\outputs\flutter-apk\app-release.apk

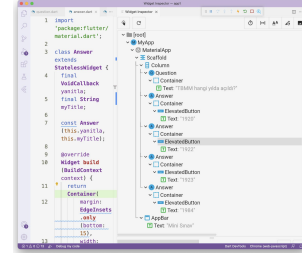
Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

2- BİLEŞENLER (Slayt 1-10)

3. Hafta: Bileşenler (Giriş)

Haftanın Kazanımları

- Bileşen (Widget) ve Bileşen Ağacı (Widget Tree) kavramını açıklayabilme
- Uygulamaya **simge (Icon)**, **metin (Text)** ve **resim (Image)** ekleyebilme
- Metin stillerini düzenleyebilme, **tema (Theme)** kullanabilme



Haftanın Amacı: Flutter uygulamalarına içerik eklemek ve bu içeriklerin sunum biçimini (stilini) düzenleyebilmek.

İşleniş: Gözle → Uygula → Tasarla → Üret → Değerlendir

3- Bileşen (Widget) ve Bileşen Ağacı (Widget Tree)

- Flutter'da ekranda görünen **her şey bir bileşendir (Widget)**: yazılar, resimler, listeler, boşluklar...
- Uygulamalar, çok sayıda bileşenin bir araya getirilmesiyle (iç içe yerleştirilmesiyle) oluşur.
- Bileşenler bir **ağaç (Widget Tree)** yapısında organize edilir. Ağacın kökünde genelde **MaterialApp** (Android tasarım dili) ya da **CupertinoApp** (iOS tasarım dili) bulunur.
- runApp()** fonksiyonu, kendisine verilen bileşeni uygulamanın **kök bileşeni** yapar.
- VS Code'da **Widget Inspector** paneliyle bir uygulamanın bileşen ağacı görsel olarak incelenebilir (Dart DevTools içinden ya da **Ctrl+Shift+P** → *Flutter: Inspect Widget*).

Örnek ağaç: **MaterialApp** → **Scaffold** → **AppBar (Text) + Column (Question, Answer...)**

4- Icon (Simge) Bileşeni

Açıklama: Flutter geniş bir simge kütüphanesi sunar (**Icons** sınıfı içinde binlerce hazır simge). Ek kurulum gerekmez.

```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      home: const Icon(
        Icons.account_circle,
        color: Colors.indigo,
        size: 50,
        semanticLabel: 'Hesap ikonu',
      ),
    );
  }
}
```

Kodun Açıklaması:

- **Icons.account_circle** → gösterilecek simgenin türü (zorunlu, pozisyonel parametre)
- **color** → simgenin rengi (opsiyonel, verilmezse varsayılan renk kullanılır)
- **size** → piksel cinsinden boyut (opsiyonel, Double)
- **semanticLabel** → ekran okuyucular için erişilebilirlik açıklaması (opsiyonel, String)

Çıktı: Ekranın ortasında **indigo renkli, 50 piksel boyutunda bir hesap ikonu** görünür.

5- Text Bileşeni

Açıklama: Ekranda görünen **tüm metinler** **Text** bileşeni ile oluşturulur. İlk parametre (metnin kendisi) zorunlu ve pozisyoneldir.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Metin bileşeni')),
        body: Text(
          "Lorem ipsum dolor sit amet, consectetur adipiscing elit.",
          maxLines: 4,
          overflow: TextOverflow.ellipsis,
        ),
      ),
    );
  }
}
```

Kodun Açıklaması:

- **maxLines** → gösterilebilecek maksimum satır sayısı
- **overflow** → satır sayısı aşılsa ne olacağını belirler (**clip**, **fade**, **ellipsis**, **visible**)
- **style** → yazı tipini düzenler (bir sonraki slaytta)
- **textAlign** → metnin yatay hizalanması

Çıktı: Başlık çubuğunda "Metin bileşeni" yazısı, gövdede ise Lorem Ipsum metni (4 satırdan uzunsa sonunda "..." ile kesilmiş hâli) görünür.

6- TextStyle ile Yazı Stillendirme

Açıklama: `TextStyle`, bir `Text` bileşeninin `style` parametresine verilerek yazının görünümünü (font, kalınlık, renk, boyut) düzenler.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Metin bileşeni')),
        body: Text(
          "Flutter",
          style: TextStyle(
            fontFamily: "Roboto",
            fontStyle: FontStyle.normal,
            fontWeight: FontWeight.w300,
            fontSize: 16,
            color: Colors.grey[800],
            height: 1.6,
          ),
        ),
      ),
    );
  }
}
```

Kodun Açıklaması:

- `fontWeight` → yazı kalınlığı (w100 çok ince → w900 çok kalın; varsayılan w400)
- `fontStyle` → normal / italik
- `fontSize` → piksel cinsinden boyut (varsayılan 14)
- `color` → yazı rengi
- `height` → satır yüksekliği (yazı boyutunun katı olarak)

Çıktı: "Flutter" yazısı, ince (w300), koyu gri renkte ve 16 piksel boyutunda gösterilir.

7- Google Fonts Paketini Kullanmak

Adım Adım:

1. Terminalde proje klasöründe şu komutu çalıştır: **flutter pub add google_fonts**
2. `main.dart` dosyasına paketi ekle: `import 'package:google_fonts/google_fonts.dart';`
3. `TextStyle` yerine `GoogleFonts.<fontAdi>()` metodunu kullan.

```
import 'package:flutter/material.dart';  
import 'package:google_fonts/google_fonts.dart';
```

```
Text(  
  "Google Fonts",  
  style: GoogleFonts.monoton(  
    fontSize: 30,  
    fontWeight: FontWeight.w300,  
  ),  
)
```

```
import 'package:flutter/material.dart';  
import 'package:google_fonts/google_fonts.dart';  
  
void main() {  
  runApp(MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('Metin bileşeni')),  
        body: Text(  
          "Google Fonts",  
          style: GoogleFonts.monoton(fontSize: 30, fontWeight: FontWeight.w300),  
        ),  
      ),  
    );  
  }  
}
```

Kodun Açıklaması: `GoogleFonts` paketi, fontları önceden indirmeden internetten çevrimiçi kullanmayı sağlar. Her font için ayrı bir metot vardır (Ör: `GoogleFonts.roboto()`, `GoogleFonts.lato()`).

Çıktı: Ekranda "Google Fonts" yazısı, **Monoton** fontuyla 30 piksel boyutunda gösterilir.

Not: Fontu proje varlığı olarak eklemek istersen (indirip `pubspec.yaml`'a tanıtarak) alternatif yol da mevcuttur, ancak `google_fonts` paketi çok daha pratiktir.

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

8- Tema (Theme) Kullanmak

Theme.of(context) ile en yakın temaya ulaşılır.

```
class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData(
        brightness: Brightness.dark,
        primaryColor: Colors.greenAccent,
        textTheme: TextTheme(
          headlineLarge: GoogleFonts.lato(
            fontSize: 60,
            fontStyle: FontStyle.italic,
            fontWeight: FontWeight.w300,
          ),
        ),
      ),
      home: MyHomePage(),
    );
  }
}
```

```
// Başka bir sayfada temayı kullanmak:
Text('Merhaba!' style: Theme.of(context).textTheme.headline1, arg)
```

Kodun Açıklaması:

- `ThemeData` → uygulamanın genel görünümünü belirleyen sınıf (`MaterialApp`'in `theme` parametresine verilir)
- `brightness: Brightness.dark` → koyu tema
- `textTheme` → metin stillerini (`headlineLarge` vb.) günceller
- `Theme.of(context)` → bileşen ağacında en yakın temayı bulur ve kullanır

Çıktı: Uygulama **koyu temada** açılır; **headlineLarge** stili uygulanan her metin **Lato fontuyla, italik ve 60 piksel** boyutunda görünür. Tek bir noktadan tüm uygulamanın görünümü değiştirilebilir.

9- Resim (Image) Bileşeni: Ağ Kaynağından Resim

İki resim ekleme yöntemi:

Yöntem	Avantaj	Dezavantaj
Proje varlığı (asset)	Her zaman erişilebilir, hızlı yüklenir	Sabittir, uygulama boyutunu artırır
Ağ kaynağı (network)	Sonradan değişebilir, boyutu artırmaz	İnternet gerektirir, yavaş olabilir

```
Image.network(
  'https://images.pexels.com/photos/40984/animal-ara-macao-beak-bird-40984.jpeg'
)
```

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('İnternet Görseli'),
        backgroundColor: Colors.blueGrey,
      ),
      body: Center(
        // Görselin ekranın tam ortasında durması ve taşmaması için
        Center içinde aldık.
        child: Image.network(
          'https://images.pexels.com/photos/40984/animal-ara-macao-beak-bird-40984.jpeg',
          // İsteğe bağlı parametreler:
          width: 300, // Genişlik
          height: 300, // Yükseklik
          fit: BoxFit.cover, // Görselin çerçeveye nasıl sığacağına
            belirler
          ),
        ),
      ),
    );
  }
}
```

Kodun Açıklaması: `Image.network()` metodu, verilen URL'deki görseli otomatik olarak indirip sayfaya ekler. `width`, `height`, `fit`, `color`, `colorBlendMode` gibi parametrelerle boyutu ve görünümünü ayarlanabilir.

Çıktı: Belirtilen URL 'deki resim ekranda otomatik olarak gösterilir.

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

10- Resim Bileşeni: Proje Varlığı (Asset) ile Renk Efekti

Adım Adım:

1. Resmi indir → proje kök klasöründe bir **resimler/** klasörü oluştur → resmi oraya koy.
2. **pubspec.yaml** dosyasında tanıt (girintiler TAB ile):

```
assets:  
  - resimler/flamingo.jpg
```

3. Kodda kullan:

```
Image.asset(  
  "resimler/flamingo.jpg",  
  color: Colors.lime,  
  colorBlendMode: BlendMode.multiply,  
)
```

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    // const kelimesi buradan kaldırıldı  
    return MaterialApp(debugShowCheckedModeBanner: false, home:  
      MyHomePage());  
  }  
}  
  
class MyHomePage extends StatelessWidget {  
  const MyHomePage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('Asset Görseli (Color Effect)'),  
        backgroundColor: Colors.blueGrey,  
      ),  
      body: Center(  
        child: Image.asset(  
          "resimler/flamingo.jpg",  
          color: Colors.lime,  
          colorBlendMode: BlendMode.multiply,  
          fit: BoxFit.cover, // Slayttaki açıklamaya uygun olarak eklendi  
        ),  
      ),  
    );  
  }  
}
```

```
name: my_flutter_app4  
description: "A new Flutter project."  
publish_to: "none"  
  
version: 1.0.0+1  
  
environment:  
  sdk: "3.12.2"  
  
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: "1.0.8"  
  google_fonts: "8.2.0"  
  
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  flutter_lints: "6.0.0"  
  
flutter:  
  uses-material-design: true  
  
# Resim dosyasını uygulamaya burada tanıtıyoruz  
assets:  
  - resimler/flamingo.jpg
```

Kodun Açıklaması: `Image.asset()` proje içine gömülmüş bir resmi gösterir. `color` + `colorBlendMode` ile resmin üzerine renk efekti uygulanabilir (`multiply`, `screen` vb.). `fit`: `BoxFit.cover` gibi parametrelerle resmin kutuya sığdırılma şekli belirlenir.

Çıktı: Flamingo resmi, üzerine **açık yeşil (lime)** rengi `multiply` yöntemiyle uygulanmış şekilde gösterilir (resim yeşilimsi bir tona bürünür).

11- Hafta 3 Özet ve Yapılacaklar (Adım Adım)

Bu hafta ne yaptık?

1. Bileşen ve Widget Tree kavramını öğrendik
2. `Icon` ile simge ekledik
3. `Text` + `TextStyle` ile metin stillendirdik
4. `google_fonts` paketiyle özel font kullandık
5. `ThemeData` ile uygulama genelinde tema tanımladık
6. `Image.network` / `Image.asset` ile resim ekledik

Tasarla-Üret-Değerlendir Görevi: Uygulaman için bir **metin teması (tipografi)** tasarla.

- Araç: typescale.com üzerinde başlık/gövde fontlarını seç
- Kontrol listesi: Başlık ve gövde fontları uyumlu mu? Renkler kontrast sağlıyor mu? Satır uzunlukları doğru mu (masaüstünde 50-60, mobilde 30-40 karakter)?

4. Hafta: Yayılım Bileşenleri ve Arayüz Tasarımı (Giriş)

Haftanın Kazanımları

- Statik arayüzler tasarlayabilme
- Flutter uygulamalarının **yayılımını (layout)** düzenleyebilme
- İçerikleri etkili biçimde sunan arayüzler oluşturabilme

İki tür kapsayıcı bileşen:

- **Tek elementli:** **Container**, **Center** → çocuk bileşen **child** parametresine verilir
- **Çok elementli:** **Row**, **Column**, **Flexible** → çocuklar bir liste hâlinde **children** parametresine verilir

13- Container ve Kutu Modeli (Box Model)

```
Container(  
  child: Icon(Icons.ac_unit, color:  
Colors.white, size: 50),  
  margin: EdgeInsets.all(30),  
  padding: EdgeInsets.all(10),  
  decoration: BoxDecoration(  
    color: Colors.lightBlue,  
    border: Border.all(color:  
Colors.lightBlueAccent, width: 2),  
    borderRadius: BorderRadius.circular(10),  
  ),  
)
```

Kodun Açıklaması (Kutu Modeli — dıştan içe):

- **Margin kutusu** → diğer bileşenlerle arasındaki boşluk (**margin**)
- **Border kutusu** → kenarlık (**decoration.border**)
- **Padding kutusu** → kenarlık ile içerik arasındaki boşluk (**padding**)
- **İçerik kutusu** → çocuk bileşenin (**child**) kendi boyutu

BoxDecoration ile artalan rengi, kenarlık ve köşe yumuşatma (**borderRadius**) ayarlanır.

Çıktı: Beyaz bir kar tanesi ikonu; açık mavi artalanlı, yuvarlak köşeli, açık mavi kenarlıklı bir kutu içinde, etrafında 30px boşlukla gösterilir.

14- Expanded Bileşeni

Açıklama: Expanded, Row/Column/Flex içinde bulunduğu tüm boşluğu doldurur. Birden fazla Expanded varsa kalan alan aralarında eşit paylaşılır.

```
Row(  
  children: [  
    Kutu(Colors.lightBlue),  
    Expanded(child: Kutu(Colors.deepPurple)),  
    Expanded(child: Kutu(Colors.teal)),  
  ],  
)
```



Kodun Açıklaması: İlk kutu sabit boyutunda kalır; diğer iki kutu Expanded içine alındığından kalan boşluğu eşit paylaşarak genişler.

Çıktı: Soldaki kutu sabit genişlikte, ortadaki ve sağdaki kutular ekranın kalan genişliğini eşit şekilde paylaşarak büyür.

15- SizedBox ve Spacer Bileşenleri

SizedBox — hem çocuğunu ölçekler hem de boş boşluk üretir:



```
Row(children: [  
  SizedBox(width: 200, height: 200, child:  
    FlutterLogo(),  
    FlutterLogo(),  
    SizedBox(width: 100), // boş boşluk  
    FlutterLogo(),  
])
```

Kodun Açıklaması: **SizedBox** sabit boyutlu kutu/boşluk üretirken, **Spacer** esnek (dinamik) boşluk üretir. **Spacer**'ın **flex** parametresiyle, birden fazla **Spacer** kullanıldığında boşluğun nasıl paylaşılacağı ayarlanabilir.

Spacer — bulunduğu alandaki tüm boşluğu kullanarak iki bileşeni birbirinden ayırır:

```
Row(children: [  
  FlutterLogo(size: 50),  
  Spacer(),  
  Icon(Icons.menu, color:  
    Colors.white, size: 55),  
])
```

Çıktı: **SizedBox** örneğinde: büyük bir logo, küçük bir logo, aralarında 100px boşluk, sonra bir logo daha. **Spacer** örneğinde: Flutter logosu solda, menü ikonu sağda, aradaki tüm boşluk otomatik dolar.

16- Row ve Column: Ana Eksen (MainAxis) ve Çapraz Eksen (CrossAxis)

```
Row(  
  mainAxisSize: MainAxisSize.max,  
  mainAxisAlignment:  
    MainAxisAlignment.spaceAround,  
  crossAxisAlignment:  
    CrossAxisAlignment.center,  
  children: [sembol, sembol, sembol, sembol],  
)
```

Kodun Açıklaması:

- **Row** → ana eksenini **yatay** (çocuklar yan yana dizilir)
- **Column** → ana eksenini **dikey** (çocuklar alt alta dizilir)
- **mainAxisSize** → **max** (tüm alanı kapla, varsayılan) / **min** (sadece içerik kadar yer kapla)
- **mainAxisAlignment** → ana ekseninde arta kalan boşluğun dağıtım şekli
- **crossAxisAlignment** → çapraz ekseninde hizalama

Çıktı: 4 simge yatayda eşit aralıklarla (**spaceAround**) dizilir ve dikeyde ortalanır.

17- MainAxisAlignment / CrossAxisAlignment Değerleri

MainAxisAlignment	Etki	CrossAxisAlignment	Etki (Row için)
<code>start</code>	Başlangıca (sola) hizalar	<code>start</code>	Üste hizalar
<code>end</code>	Sona (sağa) hizalar	<code>end</code>	Alta hizalar
<code>center</code>	Ortalar	<code>center</code>	Ortalar
<code>spaceBetween</code>	Boşluğu bileşenler arasında eşit dağıtır	<code>stretch</code>	Çapraz eksenini doldurur
<code>spaceEvenly</code>	Boşluğu bileşenler çevresinde eşit dağıtır		
<code>spaceAround</code>	Boşluğu dağıtır; ilk/son bileşenlerde yarım boşluk bırakır		

Dikkat: `mainAxisSize: MainAxisSize.min` seçildiğinde `mainAxisAlignment` etkisi gözlenemez (çünkü bileşen zaten içeriği kadar yer kaplar).

Açıklama: **Flexible**, içine alınan bileşenin boyutunu **ekran boyutuna göre esnek hâle** getirir (Responsive Design).

```
Row(  
  children: [  
    Kutu(),  
    Flexible(child: Kutu(), flex: 1, fit:  
FlexFit.loose),  
    Flexible(child: Kutu(), flex: 2, fit:  
FlexFit.loose),  
    Flexible(child: Kutu(), flex: 2, fit:  
FlexFit.tight),  
  ],  
)
```



Kodun Açıklaması:

- **flex** → alandaki payı belirler (toplam flex değerine oranla). Burada ikinci kutu %20, üçüncü %40, dördüncü %40 alan kaplar.
- **fit: FlexFit.loose** → flex alanına girer ama **ölçeklenmez**
- **fit: FlexFit.tight** → flex alanına girer ve **alana tam uyacak şekilde ölçeklenir**

Çıktı: İlk kutu sabit (50px) kalır; diğer üç kutu kalan alanı **flex** oranlarına göre paylaşır; ekran boyutu değiştikçe **tight** olan kutu(lar) yeniden ölçeklenir.

19- ListView Bileşeni

Basit kullanım (**children** ile):

```
ListView(
  padding: EdgeInsets.all(8),
  children: <Widget>[
    Container(child: Center(child: Text('Ekmek'))),
    Container(child: Center(child: Text('Süt'))),
    Container(child: Center(child: Text('Yumurta'))),
  ],
)
```



Dinamik kullanım (**builder** ile — çok sayıda eleman için):

```
var alisverisListem = ["EKMEK", "SÜT", "YOĞURT", "MAKARNA"];

ListView.builder(
  itemCount: alisverisListem.length,
  itemBuilder: (BuildContext context, int index) {
    return Container(
      padding: EdgeInsets.all(10),
      color: Colors.orange[200],
      child: Center(child: Text(alisverisListem[index])),
    );
  },
)
```

Kodun Açıklaması:

ListView.builder, **itemCount** kadar döngü ile **itemBuilder** fonksiyonunu çalıştırır; **index** değişkeni her seferinde otomatik artar. Sabit sayıda değil, **listeden gelen** eleman kadar satır üretmek için kullanılır. **scrollDirection: Axis.horizontal** ile yatay kaydırmalı liste de yapılabilir.

Çıktı: Alışveriş listesindeki 4 ürün, turuncu artalanlı kutular içinde alt alta (dikey kaydırmalı) listelenir.

20- GridView Bileşeni

Açıklama: `GridView`, `ListView`'e benzer ama elemanları **ızgara (grid)** düzeninde, belirli sütun sayısı ile gösterir.

```
GridView.count(  
    padding: EdgeInsets.all(40),  
    mainAxisSpacing: 25,  
    crossAxisSpacing: 25,  
    crossAxisCount: 4,  
    children: olustur() as List<Widget>,  
)
```

Kodun Açıklaması:

- `crossAxisCount` → satır başına kaç eleman gösterileceği (burada 4 sütun)
- `mainAxisSpacing` / `crossAxisSpacing` → elemanlar arası dikey/yatay boşluk
- `children` → genelde bir fonksiyon (Ör: `olustur()`) ile dinamik olarak üretilen `Widget` listesi verilir.

Adım Adım (resimlerle grid oluşturmak):

1. Resimleri `img/` klasörüne koy → `pubspec.yaml`'a `assets:` altında tanı
2. Resim yollarını bir `Map` içinde tut, `forEach` ile `Image.asset()` bileşenlerine çevir
3. Üretilen listeyi `GridView.count`'un `children` parametresine ver

Çıktı: Örnekte 13 sosyal medya ikonu, **4 sütunlu** bir ızgara düzeninde, aralarında eşit boşluklarla gösterilir.

Açıklama: Flutter özel "hazır" arayüz şablonları sunmaz — karmaşık arayüzler, öğrendiğimiz kapsayıcı bileşenlerin **iç içe ve bir arada** kullanılmasıyla inşa edilir.

Örnek strateji (yatay bir ürün detay ekranı için):

- En üstte bir **Row** → ekranı 2/5 - 3/5 oranında bölmek için 2 adet **Flexible**
- Sağ tarafta: resim için **Container** + **BoxFit.cover**
- Sol tarafta: **Container** (padding: 20) içinde bir **Column**
- Metinler arasında boşluk için farklı **flex** değerlerine sahip **Spacer**'lar
- Simgeleri yan yana göstermek için içeride küçük bir **Row** (**spaceEvenly**)

Önemli fikir: Karmaşık arayüzler = **basit bileşenlerin akıllıca birleşimi**. Her bileşenin görevini bilmek, en anlaşılır ve esnek arayüzü kurmayı sağlar.

22- Uygula: Dikey Arayüz Oluşturma

Görev: Yatay ekran için tasarlanmış bir arayüzü (resim + bilgi sütunları), **dikey (portre) ekrana** uygun şekilde yeniden düzenle.

Adım Adım:

1. En dıştaki **Row**'u **Column**'a çevir (ya da resmi üstte, bilgiyi altta göster)
2. **Flexible** oranlarını (2/5 - 3/5) dikey eksene göre yeniden ayarla
3. İçerideki **Row/Column** yapılarını ekran genişliğine göre kontrol et
4. Widget Inspector ile sonucu incele

Mobil uygulamalar çoğunlukla **dikey** ekranlarda çalıştığından bu alıştırma önemlidir.

Görev: Bir satın alma (pricing) ekranı tasarla.

- İncele: dribbble.com üzerindeki örnek "Pricing Page" tasarımları
- Kontrol listesi: checklist.design/pages/pricing

Değerlendirme Kriterleri:

- **Anlaşılabilirlik (clarity)** — kullanıcı ne yapması gerektiğini anlıyor mu?
- **Kısalık (conciseness)** — gereksiz bilgi var mı?
- **Kullanışlılık (usefulness)** — arayüz kullanıcıya gerçekten yardımcı oluyor mu?

24- Hafta 4 Özet ve Yapılacaklar (Adım Adım)

Bu hafta öğrendiklerimiz:

1. **Container** ile Kutu Modeli (margin - border - padding - içerik)
2. **Expanded**, **SizeBox**, **Spacer** ile boşluk/ölçekleme yönetimi
3. **Row / Column** ile yatay/dikey yayılım, **MainAxisAlignment** / **CrossAxisAlignment**
4. **Flexible** ile responsive (elastik) tasarım
5. **ListView** (**children** ve **builder**) ile listeler
6. **GridView** ile ızgara düzenleri
7. Tüm bunları birleştirerek karmaşık arayüzler kurma

Sıradaki adım: Kendi uygulaman için bir ekran tasarımı yap, kapsayıcı bileşenleri birleştirerek kodla.

5. Hafta: Etkileşimli Uygulamalar (Giriş)

Haftanın Kazanımları

- Özel (kompozit) bileşen üretebilme
- **http** istekleri ile veri çekebilme
- Birden fazla sayfadan oluşan (çok rotalı) uygulamalar üretebilme

Haftanın Amacı: Uzak bir sunucudan veri çekebilen, çok sayfalı ve etkileşimli bir Flutter uygulaması oluşturmak.

26- Kompozit Bileşenler

Açıklama: Sık kullanılan bileşen kombinasyonlarını (Ör: gradyanlı, yuvarlak köşeli, resimli kutu) **tekrar tekrar yazmamak** için kendi bileşenimizi tanımlarız.

```
class ResimKutusu extends StatelessWidget {  
  final String _yol;  
  final Color _renk;  
  final Color _renk2;  
  ResimKutusu(this._yol, this._renk, this._renk2);
```

```
@override  
Widget build(BuildContext context) {  
  return Container(  
    child: Image.asset(_yol),  
    padding: EdgeInsets.all(15),  
    margin: EdgeInsets.all(15),  
    decoration: BoxDecoration(  
      gradient: LinearGradient(  
        colors: [_renk, _renk2],  
        begin: Alignment.topLeft,  
        end: Alignment.bottomRight,  
      ),  
      borderRadius: BorderRadius.circular(15),  
    ),  
  );  
}
```

Kodun Açıklaması: **ResimKutusu**, kendi **StatelessWidget** sınıfımızdır. Yapıcı metodu (**ResimKutusu(this._yol, this._renk, this._renk2)**) 3 parametre alır. **final** ile tanımlanan değişkenler oluşturulduktan sonra değişmez. **LinearGradient** ile köşeden köşeye renk geçişi (gradyan) oluşturulur.

Kullanımı:

```
ResimKutusu("resimler/meyveler/limon.png", Colors.purple,  
Colors.purpleAccent)
```

Çıktı: Sol üstten sağ alta mor tonlarında gradyan geçişli, köşeleri yuvarlatılmış, içinde limon resmi olan bir kutu görünür.

Adım Adım (Model Sınıfı + map() ile):

```
// 1) Adım: Veri modeli sınıfı oluştur
class KutuModeli {
    final String _yol;
    final Color _renk1;
    final Color _renk2;
    KutuModeli(this._yol, this._renk1, this._renk2);
}

// 2) Adım: Model nesnelerinden bir liste oluştur
final List<KutuModeli> meyvelerim = [
    KutuModeli("resimler/meyveler/limon.png", Colors.purple, Colors.purple),
    KutuModeli("resimler/meyveler/cilek.png", Colors.orange, Colors.orange)
];

// 3) Adım: map() ile listeyi ResimKutusu bileşenlerine çevir
GridView.count(
    crossAxisCount: 2,
    children: meyvelerim.map((e) {
        return ResimKutusu(e._yol, e._renk1, e._renk2);
    }).toList(),
)
```

Kodun Açıklaması: `map()` metodu, listedeki **her bir elemanı** anonim fonksiyona gönderir ve dönen değerlerden yeni bir liste (`Iterable`) oluşturur. `.toList()` ile bu `Iterable`, `GridView`'in beklediği `List<Widget>` türüne çevrilir. Bu yöntem, **sayısı bilinmeyen** (Ör: internetten gelen) veriler için idealdir.

Çıktı: 2 sütunlu bir ızgarada, `meyvelerim` listesindeki her eleman için otomatik olarak bir `ResimKutusu` oluşturulur.

28- Durum (State) Kavramı

Flutter **tanımlayıcı (declarative)** bir platformdur: veri değiştiğinde arayüz **otomatik olarak yeniden çizilir** (imperative dillerdeki gibi `Label1.Text = "..."` şeklinde manuel güncelleme yapılmaz).

State, uygulama düzeyinde (App State — Ör: kullanıcı girişi) veya bileşen düzeyinde (Widget State — Ör: bir sayacın değeri) olabilir.

StatelessWidget: içeriği hiç değişmeyen bileşenler (Ör: sabit bir `Text`).

StatefulWidget: içeriği zamanla değişebilen, kendini güncelleyebilen bileşenler.

Durumlu (Stateful) bir bileşen 2 sınıftan oluşur:

Sınıf	Görevi
<code>StatefulWidget</code>	Arayüzü temsil eder
<code>State</code>	Değişebilen veri yapısını tutar, <code>setState()</code> çağrıldığında arayüzü günceller

VS Code ipucu: `st` yazıldığında çıkan öneri panelinden **Flutter StatefulWidget** seçilerek iskelet kod otomatik oluşturulabilir.

29- StatefulWidget ve State Sınıfının Bağlanması

```
import "package:flutter/material.dart";

void main() {
  runApp(DurumluUygulamam());
}

class DurumluUygulamam extends StatefulWidget {
  @override
  _DurumluUygulamamState createState() {
    return _DurumluUygulamamState();
  }
}

class _DurumluUygulamamState extends State<DurumluUygulamam> {
  var _tur = 0;
  var _son = DateTime.now();
  List<String> _satirlar = [];

  void basildi() {
    setState(() {
      // arayüzü etkileyecek değişiklikler burada yapılır
    });
  }

  Widget build(BuildContext context) {
    return MaterialApp(/* ... */);
  }
}
```

Kodun Açıklaması:

- **DurumluUygulamam** (StatefulWidget) → **createState()** ile kendine bağlı **_DurumluUygulamamState** nesnesini üretir
- **_DurumluUygulamamState** **extends State<DurumluUygulamam>** → bu bağı iki sınıf arasında kurar
- Sınıf içindeki **_tur**, **_son**, **_satirlar** değişkenleri **veri yapısını** oluşturur
- **setState(() { ... })** çağrılan her yerde, içindeki kod çalışır **ve arayüz otomatik olarak yeniden çizilir**

30- Tur Sayacı Uygulaması: setState() ile Arayüz Güncelleme

```
void basildi() {
    var simdi = DateTime.now();
    var sure = Duration(
        milliseconds: (simdi.difference(_son).inMilliseconds))
        .toString()
        .substring(0, 11);
    _son = simdi;
    _tur++;
    setState() {
        _satirlar.add("$_tur. Tur: $sure sn");
    });
}
```

İlgili buton:

```
ElevatedButton(
  onPressed: basildi,
  style: ElevatedButton.styleFrom(
    const CircleBorder()),
  child: const Icon(Icons.add, size:
```

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DurumlUygulamam());
}

// GÖRSEL 27- VERİ MODELİ VE LİSTE TANIMLAMASI
//
class KuluModel {
  final String _yol;
  final Color _renk1;
  final Color _renk2;
  KuluModel(this._yol, this._renk1, this._renk2);
}

final List<KuluModel> meyvelerim = [
  KuluModel('resimler/meyveler/limon.png',
    Colors.purple, Colors.purpleAccent),
  KuluModel('resimler/meyveler/çilek.png',
    Colors.orangeAccent, Colors.yellow),
];

//
// GÖRSEL 29- STATEFUL WIDGET YAPISI
//
class DurumlUygulamam extends StatefulWidget {
  const DurumlUygulamam({super.key});

  @override
  _DurumlUygulamamState createState() {
    return _DurumlUygulamamState();
  }
}

class _DurumlUygulamamState extends
  State<DurumlUygulamam> {
  // Durum deęiklerimiz (State)
  int _tur = 0;
  DateTime _son = DateTime.now();
  final List<String> _satirlar = [];

  //
  // GÖRSEL 30- TUR SAYACI VE SÜRE HESAPLAMA
  //
  //
  void basildi() {
    var simdi = DateTime.now();
    var sure = Duration(
      milliseconds: (simdi.difference(_son).inMilliseconds),
    ).toString().substring(0, 11);

    _son = simdi;
    _tur++;

    setState() {
      _satirlar.add("$_tur. Tur: $sure sn");
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Kompozit Bileşenler & Tur Sayacı'),
          backgroundColor: Colors.blueGrey,
        ),
        body: SafeArea(
          child: Column(
            children: [
              // 1) Dinamik GridView Alanı (Görset 27'deki
              // map kullanimi)
              SizedBox(
                height: 180,
                child: GridView.count(
                  crossAxisCount: 2,
                  padding: const EdgeInsets.all(10.0),
                  children: meyvelerim.map((e) {
                    return ResimKutusu(e._yol, e._renk1,
                      e._renk2);
                  }).toList(),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Kodun Açıklaması:

1. **DateTime.now()** ile şu anki zaman alınır
 2. **simdi.difference(_son)** ile önceki tur ile aradaki fark (milisaniye) hesaplanır
 3. **_son** güncellenir, **_tur** sayacı bir artırılır
 4. Yeni satır **setState()** içinde **_satirlar** listesine eklenir → bu, arayüzdeki **ListView.builder**'ın otomatik olarak yeniden çizilmesini te
- Dikkat: **onPressed: basildi** yazılır — parantez konulmaz (**basildi()** değil)! Parantez konursa fonksiyon hemen çalışır, buton her bası

Çıktı: Her butona basışta yeni bir "N. Tur: XX.XXX sn" satırı listeye eklenir ve ekranda anında görünür.

Neden gerekli? Günümüzde neredeyse tüm uygulamalar (sosyal ağlar, alışveriş) veriyi **dışarıdan** (bir sunucudan) çeker.

JSON (JavaScript Object Notation) — uygulamalar arası veri aktarımında kullanılan standart format:

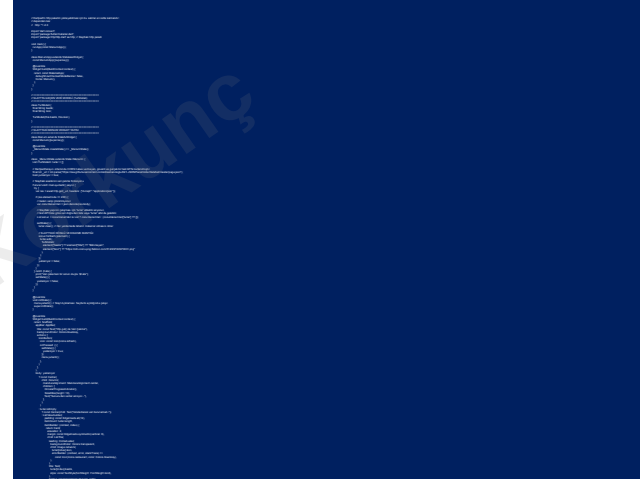
```
{ "turler":  
  [  
    {  
      "id": 1,  
      "baslik": "Balıklar",  
      "yemekler": ["Balık Çorbası", "Levrek Izgara"],  
      "ikon": "https://.../balik.png"  
    }  
  ]  
}
```

Kuralları:

- Nesneler { }, listeler [] içinde tanımlanır
- String değerler ve anahtar isimleri **tırnak içinde**, sayılar/boolean tırnaksız yazılır
- Anahtar : değer çiftleri iki nokta (:) ile ayrılır

32- http.get() ile Veri Çekmek (async / await)

```
class _MenuState extends State<Menu> {  
  List<TurModeli> turler = [];  
  final Uri _url = Uri.parse("https://my-json-server.typicode.com/.../db");  
  
  Future<void> menuyuGetir() async {  
    var res = await http.get(_url, headers: {"Accept": "application/json"});  
    List sonuc = json.decode(res.body)["turler"];  
    setState(() {  
      sonuc.forEach((element) {  
        turler.add(TurModeli(element["baslik"], element["ikon"]));  
      });  
    });  
  }  
  
  @override  
  void initState() {  
    menuyuGetir();  
    super.initState();  
  }  
}
```



- Gelen veriler `setState()` içinde listeye eklenerek arayüz güncellenir

Gerekli paketler: `import`

`'package:http/http.dart' as http;` ve `import 'dart:convert';`

Çıktı: Uygulama açılır açılmaz sunucudan veri çekilir; `turler` listesi doldukça arayüz otomatik güncellenir.

33- Çekilen Veriyi GridView'de Göstermek

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text("Deneyap
Restoran")),
    body: GridView.count(
      padding: const EdgeInsets.all(30),
      mainAxisSpacing: 20,
      crossAxisSpacing: 20,
      crossAxisCount: 2,
      children: [...turler.map((e) =>
(Tur(e._baslik, e._img))).toList()],
    ),
  );
}
```



İnternette
 sindeki her
 r **Tur**
 e dönüştürölüp
 Böylece **kaç**
ceden
 ız otomatik

len her yemek
 arada resim +
 enir.

34- Çok Sayfalı Uygulamalar: Rota (Route) Kavramı

Flutter'da her bağımsız sayfaya **rota (route)** denir.

Navigator, gösterilen rotaların bir **yığının (stack)** tutan bileşendir:

- `Navigator.push(...)` → yığına yeni bir sayfa **ekler** (ileri git)
- `Navigator.pop(context)` → yığının en üstündeki sayfayı **çıkartır** (geri dön)

```
IconButton(  
  icon: const Icon(Icons.pie_chart_outline),  
  onPressed: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(builder: (context) =>  
const GrafikSayfam()),  
    );  
  },  
)
```

Kodun Açıklaması: `MaterialPageRoute`, yeni sayfanın nasıl oluşturulacağını (`builder`) tanımlar. Yeni sayfaya geçildiğinde, `AppBar` üzerinde otomatik bir **geri butonu** oluşur — bu buton `Navigator.pop()`'u kendiliğinden çağırır.

Çıktı: Grafik ikonuna basıldığında ekran, `GrafikSayfam` sayfasına geçer; sol üstteki geri okuna basınca önceki sayfaya dönlür.

35- Örnek: pub.dev Paketiyle Grafik Sayfası

Adım Adım (3. parti paket eklemek):

1. `pubspec.yaml` → `dependencies:` altına ekle: `pie_chart: #latest versiyon`
2. Terminalde `flutter pub get` çalıştır (ya da otomatik güncellenir)
3. Kodda import et: `import 'package:pie_chart/pie_chart.dart';`

```
PieChart(  
  dataMap: harcamalar,      // Map<String, double>  
  chartType: ChartType.ring,  
  chartRadius: MediaQuery.of(context).size.width / 2,  
)
```



Kodun Açıklaması: `pub.dev`, Flutter/Dart topluluğunun paylaştığı **ücretsiz paketlerin** bulunduğu resmi ambardır (Ör: grafik, tarih formatlama, mesaj balonu paketleri). `PieChart` bileşeni, verilen `Map` verisinden otomatik olarak pasta grafiği çizer.

Çıktı: `harcamalar` haritasındaki her kategori, kendine ait renkte bir dilim olarak halka (ring) tipinde pasta grafiğinde gösterilir.

36- Drawer (Çekmece) Bileşeni

Açıklama: Uzun menüleri gizli tutup gerektiğinde **drawer** (sol) veya **endDrawer** (sağ) parametrelerle açılır.

```
class Cekmece extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Drawer(  
      child: ListView(  
        children: [  
          const UserAccountsDrawerHeader(  
            accountName: Text("@odonmez"),  
            accountEmail: Text("onur.donmez@ege.edu.tr"),  
            currentAccountPicture: CircleAvatar(  
              backgroundImage: AssetImage("img/account.jpg"),  
            ),  
          ),  
          ListTile(title: const Text("Giriş"), leading: const Icon(Icons.home)),  
          ListTile(title: const Text("Sepetim"), leading: const Icon(Icons.shopping_cart)),  
        ],  
      ),  
    );  
  }  
}  
  
// Kullanımı:  
Scaffold(drawer: const Cekmece(), /* ... */)
```

hamburger menüsüdür. **Scaffold**'un **drawer** parametresi ile kullanılır.

AccountsDrawerHeader Açıklaması:

AccountsDrawerHeader kullanıcı bilgilerini (ad, e-posta, resim) gösteren özel bir başlık alanıdır. **ListTile** widgetinin **leading** + simge içeren, tıklanabilir bir öğe satırıdır (**leading**: ön simge, **title**: başlık, **onTap**: tıklama olayı).

Ekranın solundan kaydırılarak (ya da **Scaffold**'daki hamburger ikonuna basılarak) açılır. İçerisinde bir menü; üstte hesap bilgileri, altında "Giriş" ve "Sepetim" seçenekleri görünür.

37- İsimli Rotalar (Named Routes)

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const GirişSayfasi(),  
    '/sepetim': (context) => const SepetimSayfasi(),  
    '/ayarlar': (context) => const AyarlarSayfasi(),  
  },  
)
```

// Drawer içindeki bir ListTile'dan yönlendirme:

```
ListTile(  
  title: const Text("Ayarlar"),  
  onTap: () => Navigator.pushNamed(context,  
    'ayarlar'),  
)
```

Ko

uyg

isn

çağ

ha

in

gö

pa

Na

' /

Çı

tıkl

/a

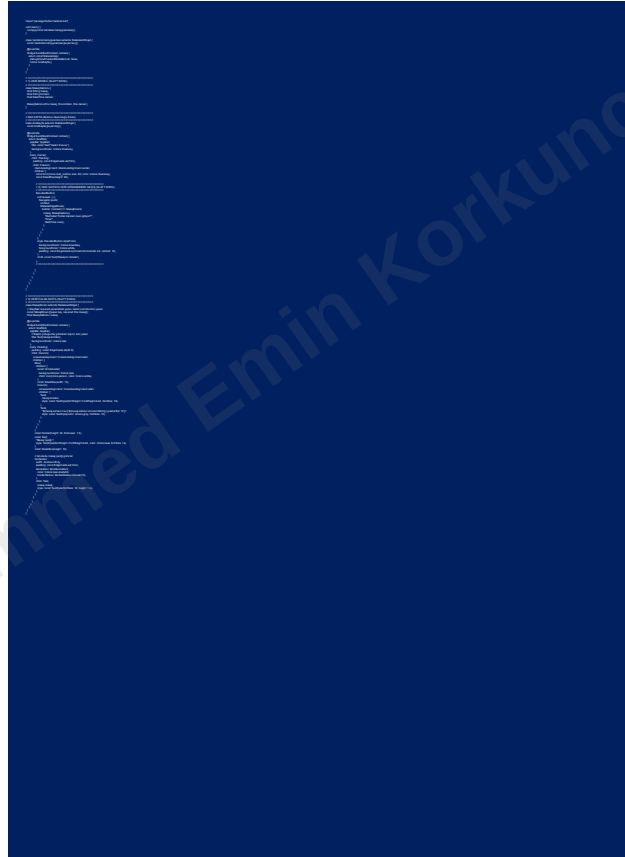
38- Rotalara Veri Aktarmak (Yapıcı Metot ile)

```
// 1) Veri modeli
class MesajSablonu {
  final String mesaj;
  final String kimden;
  final DateTime zaman;
  MesajSablonu(this.mesaj, this.kimden, this.zaman);
}

// 2) Yeni sayfaya veri göndererek geçiş
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => MesajEkrani(
          mesaj: MesajSablonu("Mesaj içeriği.", "Onur", DateTime.now()),
        ),
      ),
    );
  },
  child: const Text("Mesajımı göster"),
)

// 3) Veriyi alan sayfa
class MesajEkrani extends StatelessWidget {
  const MesajEkrani({Key? key, required this.mesaj}) : super(key: key);
  final MesajSablonu mesaj;

  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(mesaj.kimden)),
      body: Text(mesaj.mesaj),
    );
  }
}
```



hedef bileşenin
red this.mesaj)

r.

builder

fa, ihtiyacı olan

Bu yöntem sınıflar
nilir veri aktarımı

MesajEkrani

Onur" yazar, gövdede

39- İsimli Rotalara Veri Göndermek (arguments ile)

```
// Gönderen taraf:
ElevatedButton(
  onPressed: () {
    Navigator.pushNamed(
      context,
      "/mesajlar",
      arguments: MesajSablonu("Merhaba", "Onur", DateTime.now()),
    );
  },
  child: const Text("Mesajımı göster"),
)

// Alan taraf:
class MesajEkrani extends StatelessWidget {
  const MesajEkrani({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    final mesaj = ModalRoute.of(context)!.settings.arguments as
    MesajSablonu;
    return Scaffold(
      appBar: AppBar(title: Text(mesaj.kimden)),
      body: Text(mesaj.mesaj),
    );
  }
}
```



İsimli rotalarda yapıcı
mekmek pratik değildir; bunun
pushNamed() 'in
metresi kullanılır. Hedef

(context)!.settings.a
okunur; as
e bu verinin türü belirtilir (tür

ekrani, bu kez isimli rota
rinden ve arguments
nuç görsel olarak bir önceki

Görev: Basit bir **mesajlaşma uygulaması** tasarla.

- En az **3 farklı sohbet** (Ör: Onur–Beril, Onur–Fırat, Onur–Ezgi)
- Her sohbette **her iki taraftan** en az 3 mesaj olsun
- Mesaj verilerini bir **liste** hâlinde, mesajların gösterileceği rotaya aktar
- Ekran sahibinin (Ör: Onur) mesajları **yeşil**, karşı tarafinkiler **gri** gösterilsin

Değerlendirme: Anlaşılabilirlik (clarity), kısalık (conciseness), kullanışlılık (usefulness) kriterlerine göre kendi ve arkadaşlarının tasarımlarını karşılaştır.

41- 3 Haftanın Genel Özeti

- 3. Hafta — Bileşenler:** `Icon`, `Text`, `TextStyle`, `google_fonts`, `Theme`, `Image` (network/asset)
- 4. Hafta — Yayılım Bileşenleri:** `Container` (kutu modeli), `Expanded`, `SizedBox`, `Spacer`, `Row/Column` (`MainAxisAlignment`/`CrossAxisAlignment`), `Flexible`, `ListView`, `GridView`
- 5. Hafta — Etkileşimli Uygulamalar:** Kompozit bileşenler, `State/StatefulWidget/setState()`, `http.get()` + JSON, çok sayfalı gezinim (`Navigator`), `Drawer`, isimli rotalar, rotalara veri aktarma

Dersi bitirirken izlenecek adımlar:

1. Her haftanın "Gözle" bölümündeki temel kavramı bir cümleyle özetle
2. Her bölümdeki **bir** kod örneğini VS Code'da birlikte çalıştır, çıktısını göster
3. "Uygula" alıştırımlarından birini sınıfla birlikte tamamla (Ör: Flexible ile arayüz, Tur Sayacı)
4. "Tasarla-Üret-Değerlendir" görevlerini (tipografi teması, satın alma ekranı, mesajlaşma uygulaması) ödev olarak ver
5. Öğrencilere haftalık öz değerlendirme formlarını hatırlat